

Ruipu's Final Evaluation Review.

No-Technical Comments:

I am still turning my head, shaking and sometimes putting my hands on my face.

At the beginning of the interview, 1/3 of my head was not in the camera.

It was at least 2 times that I wanted to laugh. It was not about something else, but just could not control my emotion.

One reason for uncontrolled behavior might be that I was extremely tired and exhausted at that time for lack of sleep the night before yesterday.

I should always adjust my body status in future's interview. Finally, I should also be well dressed. I do not have a suit currently but I will buy one for future's interview.

Difference between interface and abstract class in Java?

Comment: Not well structured. Language is not fluent. I should at the front mention "What are they". If time permits, I should also mention what are Consumer, Producer, Predicate and Function, with one sentence for each one.

Modified Answer: Both an interface and an abstract class are inheritance mechanisms in Java. They are both used for abstraction. They help define a common contract for different classes.

An interface mainly defines what a class should do. It is a good choice when different classes share the same behavior but do not have the same parent class. By default, a Java class can extend only one parent class. However, one class can implement multiple interfaces. This gives Java a way to support multiple inheritance of behavior.

Before Java 8, an interface could contain only abstract methods. It could not have concrete methods. Since Java 8, an interface can also have default methods with implementations. It can also have static methods.

An abstract class defines what a class should do. It can also define how some parts work. It can have both abstract methods and concrete methods. It can also have instance fields, constructors, and state. A class can extend only one abstract class because Java allows only single class inheritance.

In general, I use an interface to define capabilities or contracts. I use an abstract class when related classes share common fields, common state, or common implementation.

Can you explain the difference between HashMap, ConcurrentHashMap, and TreeMap?

Comments: I should also (might briefly) mention the internal mechanism of Map interface, which is the hashCode calculation and the bucket to store.

HashMap, ConcurrentHashMap, and TreeMap are all implementations of the Map interface. They all store data as key-value pairs.

HashMap does not maintain any ordering. HashMap uses hashing internally. Each key has a hashCode method. HashMap calls this method on the key. HashMap uses the hashCode to calculate a bucket index. HashMap stores the key-value pair in that bucket. Two different keys can produce the same bucket index. This is called a collision. HashMap handles collisions with a linked list inside the bucket. If the list grows too long, HashMap converts it to a tree structure. This keeps lookups fast even with many collisions.

ConcurrentHashMap is designed for multi-threaded environments. It allows multiple threads to read and write safely. ConcurrentHashMap is mainly used in multi-threaded applications.

TreeMap keeps the keys in their natural order. Natural order comes from the Comparable interface. The key class must implement Comparable. Comparable defines a compareTo method. TreeMap uses compareTo to decide the order of the keys. We can also provide a Comparator to customize the ordering. A Comparator is a separate class. We do not need to change the key class to use a Comparator. If we pass a Comparator, TreeMap uses it instead of compareTo.

How do you design an immutable class?

Comment: Deep Copy should be more detailed and the getter method should be public. Before the interview, I am always mixing the immutable class with the singleton class. I should always memorize them together to avoid the mixture.

First, I make the class final so other classes cannot extend it. Then I make all fields private and final. This prevents the fields from being reassigned. I do not provide setter methods. I only provide public getter methods.

If the class contains mutable nested objects, I copy them in two places. In the constructor, I copy the object before assigning it to the field. This prevents outside code from changing the field through the original reference. In the getter, I return a copy of the field instead of the original reference. This prevents the caller from modifying the internal state directly.

How do you implement the Singleton pattern?

Comment: Wrong in private constructor and static getInstance. I should also figure out the one instance at the beginning. I also forgot the volatile keyword.

The Singleton is a creational design pattern that guarantees a class has exactly one instance for the whole application, and gives you a single global access point to it.

I first let the single instance be private and static field. I also implement it by making the constructor private so nobody can call new from outside. I established a static getInstance method.

Eager loading

With eager initialization, I create the single instance as soon as the class is loaded, before anyone actually asks for it. I do this by initializing the static instance field at the point where I declare it, so the instance is built immediately when the class is first loaded into the JVM.

The getInstance method then simply returns that already-created instance every time. The big advantage is that it's very simple and inherently thread-safe, because the instance is created once during class loading, before any threads can compete for it, so I don't need any locking at all.

The trade-off is that the instance gets created whether or not it's ever used, which wastes resources if it turns out nobody needs it, and it isn't ideal when creating the object is expensive.

Lazy loading

With lazy initialization, I don't create the instance up front. Instead I leave the static instance field empty, and I only create the object the first time getInstance is actually called.

Inside getInstance I check whether the instance is still null, and if it is, I create it; otherwise I just return the one that already exists. The advantage is that I save resources by never creating the instance if it's never needed, and I defer any expensive setup until it's actually required.

The catch is that in a multithreaded environment this isn't safe on its own, because two threads could both find the instance null and both create one, so I have to make it thread-safe, typically with double-checked locking and a volatile field, which is where the synchronized block and the second null check come in.

In a multithreaded environment I use lazy loading with double-checked locking with a volatile field. I check for null outside the lock for performance, so that once the instance exists every later call returns it without paying the cost of acquiring the lock. I also check the null inside the lock for correctness. That way, if two threads both pass the first check, only one of them actually creates the instance.

The `volatile` keyword is used to guarantee visibility and prevent instruction reordering.

In Singleton double-checked locking, it makes sure that once one thread creates the instance, other threads can see the updated value immediately. It also prevents other threads from seeing a half-created object.

Typical uses are database connection pools, configuration objects, loggers, and caches. The main drawback is that it introduces global state, which makes unit testing harder, and that's exactly why many teams prefer dependency injection instead.

In Spring, beans are singletons by default, but they're managed by the container, which is a cleaner approach.

design patter. Singleton, Builder, Factory, Commander.

How do you handle concurrent programming in Java?

I handle concurrency in Java using the `java.util.concurrent` package. I avoid raw threads in production code.

Thread management. I use `ExecutorService` for thread pools. I avoid creating threads manually. Manual thread creation is expensive. It wastes system resources. For production systems, I use `ThreadPoolExecutor` directly. This gives me control over core pool size. It gives me control over max pool size. It gives me control over the queue type. It gives me control over the rejection policy. Factory methods like `newFixedThreadPool` are convenient. But they can cause memory issues. Their queues are often unbounded.

Thread safety. Race conditions happen when threads share mutable state. A simple example is `count++`. This looks like one operation. It is actually three steps. The thread reads the value. The thread adds one. The thread writes it back. Two threads can interleave these steps. This causes lost updates. I prevent this with locks or atomic classes.

Locking tools. I use `synchronized` for simple cases. It is built into the JVM. It is easy to use. I use `ReentrantLock` for more control. It supports `tryLock`. It supports timeouts. It supports fair locking. Fair locking prevents thread starvation. I choose `Lock` when I need flexibility. I choose `synchronized` when the logic is simple.

Concurrent utilities. I use `ConcurrentHashMap` for shared maps. It allows safe concurrent access. It performs better than a synchronized `HashMap`. I use `AtomicInteger` for counters. Atomic classes use CAS internally. CAS stands for compare-and-swap. It avoids locking

entirely. I use `CountDownLatch` to wait for multiple tasks. I use `CompletableFuture` for async chains. It lets me combine results without blocking.

Risk management. I watch for race conditions. I watch for deadlocks. I watch for thread starvation. I prevent deadlocks with a fixed lock order. All threads acquire locks in the same sequence. I size thread pools based on the workload. CPU-bound tasks need fewer threads. I/O-bound tasks need more threads.

Comment: I should start from customizing the thread, including seven parameters and using the `ThreadPoolExecutor`. Then I should talk about 4 ways of building thread: Extending thread, Implement `Runnable`, `Callable`, `CompletableFuture`. I should also talk about race condition and then, CAS and other thread-safe data structures.

Tell me about Spring Framework.

Comment: Lack of the bean type. I did not figure out much clear about the bean (What is bean, how spring manages it, and bean type.). When I say it is more annotation driven, I did not say much about `@Controller`, `@Service` etc. I should show more examples.

Spring Framework is a lightweight Java framework used to build enterprise applications. It provides many core features (module) such as IoC (Inversion of Control), Dependency Injection, AOP, transaction management, and integration with different technologies.

IoC, or Inversion of Control, is a design principle in Spring where the control of creating and managing objects is transferred from the developer to the Spring container. + `loose coupling`

Developers do not have to create objects using new keywords.

DI is the implementation of IoC + definition+ bean types (`@Scope`)

Three ways of injection. Constructor injection is the most recommended approach. The dependency must be provided when the object is created, so the object cannot exist without it. This follows the fail-fast principle. It also avoids `NullPointerException`, supports `final` fields.

It is the easiest to unit test because we can pass mock objects through the constructor.

Field injection is the simplest to write. We only need to add `@Autowired` on the field. However, it hides dependencies, does not support `final` fields. It is difficult to unit test because we cannot easily inject mock objects.

Setter injection is useful when a dependency is optional or needs to be changed later. However, the dependency may be null if the setter is not called, it does not support `final` fields, and it is less safe than constructor injection.

AOP, or Aspect Oriented Programming, is a programming paradigm that allows you to separate cross-cutting concerns.

The concerns includes logging, security, and transaction management from the main business logic by applying them automatically across multiple points in the application. +Global Exception Handling and `@Aspect`

In the early days of Spring Framework, applications were mainly configured using XML files.

The Spring container managed objects through `BeanFactory` and `ApplicationContext`. `BeanFactory` is the basic IoC container, while `ApplicationContext` is a more advanced container and is more commonly used in modern applications.

At that time, Spring development was not annotation-driven, and developers had to define most configurations in XML.

Later, Spring MVC introduced annotations such as `@Controller`, `@RequestMapping`, and `@Autowired`, which significantly reduced the amount of XML configuration and made development easier and more maintainable.

Today, Spring Boot is built on top of Spring Framework and follows an annotation-driven development style.

Developers can use annotations such as `@SpringBootApplication`, `@RestController`, `@Service`, and `@Repository` to quickly build applications with minimal configuration.

Spring Boot also provides auto-configuration, starter dependencies, and embedded servers, which greatly simplify application development.

In summary, Spring Framework provides the foundation and core features, Spring MVC introduced annotation-based web development, and Spring Boot further simplified development through convention over configuration and auto-configuration.

What is the difference between Spring Boot and Spring Framework?

Comment: Not well Structured and not confident about what I said.

Spring Framework and Spring Boot solve different problems.

Spring Framework is the core framework. It provides IoC and dependency injection. It provides AOP. It requires manual configuration. Developers write XML config. Developers write Java config classes. This setup takes time.

Spring Boot is built on top of Spring Framework. It follows convention over configuration. It uses auto-configuration. Auto-configuration scans the classpath. It detects the libraries in the project. It configures beans automatically based on what it finds. Developers do not write most of the boilerplate config.

Spring Boot also provides starter dependencies. Starters bundle related libraries together. Starters also lock compatible versions. This prevents version conflicts. We just add one starter to `pom.xml`. We do not manage each library version by hand.

Spring Boot includes an embedded server. Tomcat is the default. This removes the need for external server setup. We can run the application as a standalone jar.

In short, Spring Framework gives us the building blocks. Spring Boot gives us a faster way to assemble them.

What is Spring Boot Actuator?

Spring Boot Actuator is used to monitor (status) and manage applications in production environments. It provides many built-in endpoints that expose application health, metrics, beans, caches, environment information, and other runtime details. To use Spring Boot Actuator, the first step is to add the actuator dependency to the project. The second step is to expose the actuator endpoints through configuration. For example, in `application-dev.properties`, we can configure: `management.endpoints.web.exposure.include=*` This allows access to endpoints such as `/actuator/health`, `/actuator/beans`, `/actuator/caches`, `/actuator/env`, and `/actuator/metrics`. The third step is to collect and persist application metrics. In a production environment, actuator metrics are commonly exported to a time-series database such as Prometheus. The fourth step is to connect Prometheus to a visualization tool such as Grafana. Grafana reads metrics from Prometheus and provides dashboards for monitoring application performance, memory usage, CPU usage, request counts, response times, and other operational metrics.

How do you manage the Spring Bean lifecycle? What Bean scopes does Spring provide?

Comment: I should talk about both questions together. I forgot the `@Scope`, Injection by name, or by type, `@Qualifier` and `@Primary`

Spring manages the full lifecycle of a bean. The lifecycle has several stages.

First, Spring creates the bean instance. It calls the constructor. Then Spring injects dependencies. It sets the properties. It sets the autowired fields.

Next, Spring runs the initialization phase. If the bean implements `Aware` interfaces, Spring calls those methods first. Then Spring runs `BeanPostProcessor` before-initialization logic. Then Spring calls `@PostConstruct` methods. Then Spring runs `BeanPostProcessor` after-initialization logic. AOP proxies are often created at this final step.

After initialization, the bean is ready to use. The application calls the bean as needed.

When the container shuts down, Spring destroys the bean. It calls `@PreDestroy` methods. It calls any custom destroy method.

I prefer `@PostConstruct` and `@PreDestroy` over `InitializingBean` and `DisposableBean`. These annotations do not couple the code to Spring interfaces. This makes the code cleaner.

I also consider bean scope. Singleton beans go through the full lifecycle. Prototype beans do not. Spring does not manage the destruction of prototype beans. The application must handle that manually.

Singleton creates only one instance of the bean that is shared across the entire Spring boot application and is the default scope in Spring. Prototype creates a new instance every single time the bean is requested, meaning there is no shared instance. Request creates a new instance for each individual HTTP request and is destroyed once the request is completed, making it specific to web applications. Session creates a new instance for each HTTP session and lives as long as the user's session is active, also specific to web applications. Application scope creates one object for the entire web application. All users share the same object. All sessions also share the same object. The object lives as long as the web application is running. It is similar to a Singleton bean, but its scope is the web application instead of the Spring application context.

How would you build a microservice architecture?

Comment: Forgot most of the components of the microservice.

I begin by identifying business capabilities and domain boundaries. I do not divide services only by technical layers such as controller, service, and repository. Instead, I divide them by business responsibility, such as customer management, order management, inventory management, payment, and notification. Each service should have a clear responsibility and own its business logic and data. I define APIs or event contracts for communication between services. I then decide which interactions should be synchronous and which should be asynchronous. I also design for failure from the beginning. Every remote call should have a timeout. Depending on

the operation, I may add retries, a circuit breaker, fallback behavior, rate limiting, and bulkhead isolation. I include centralized logging, distributed tracing, metrics, authentication, authorization, configuration management, and service discovery. Finally, I avoid making services too small without a business reason. A service should represent a meaningful business capability. Poorly chosen service boundaries can create excessive network communication and distributed transactions.

One note: that Microsoft Learn citation at the end looks like it got pulled in from another source and doesn't fit the voice of a spoken interview answer. You may want to drop it before using this in an actual interview.

What is Service Discovery?

Comment: Should speak out, what is it? And then give the example of Eureka.

Service discovery is a mechanism for microservices to find each other. In a distributed system, services run on dynamic addresses. Instances scale up and down. Instances restart with new IP addresses. Hardcoding addresses does not work in this environment.

We used Eureka. Eureka is a service registry. Every microservice registers itself with Eureka. Each instance sends its address and metadata during registration.

Eureka monitors the status of all services. Eureka continuously receives heartbeat signals. If it does not receive heartbeats for some time, it marks the service as unavailable. If one service crashes, Eureka detects it this way. Then other services know that this instance is down.

When one service needs to call another, it does not use a hardcoded URL. It queries Eureka for the target service name. Eureka returns the available instances. This is often combined with client-side load balancing. The calling service picks one instance and sends the request.

This design makes the system more resilient. Services can scale without manual configuration changes. Failed instances are removed automatically. New instances are discovered automatically.

What databases have you used before? What is the difference between SQL and NoSQL? When will you use SQL, when use NoSQL?

Comment: My answer only contains small parts. I should always start from what are they? SQL database is..., NoSQL is ... I should also talk about some famous database, including some of those on AWS. Also, my answer should combine all the three questions; that is Q"What database have you used before?", then I should answer what are they, what are the difference, when to use which, famous database examples.

I have used both SQL and NoSQL databases.

SQL databases store data in tables. Each table has rows and columns. Each table follows a fixed schema. SQL databases enforce relationships between tables. They use foreign keys. They support ACID transactions. This means data stays consistent and reliable. Common SQL databases include MySQL, PostgreSQL, and Oracle. I have mainly used MySQL and PostgreSQL. On AWS, I have used RDS. RDS supports MySQL and PostgreSQL as managed services. AWS also offers Aurora. Aurora is a cloud-native database. It is compatible with MySQL and PostgreSQL.

NoSQL databases store data differently. They do not require a fixed schema. They do not always use tables. NoSQL databases fall into different types. Document databases store data as JSON-like documents. MongoDB is a common example. Key-value stores save data as simple key-value pairs. Redis is a common example. Column-family databases store data in wide columns. Cassandra is a common example. Graph databases store data as nodes and relationships. Neo4j is a common example. On AWS, I have used DynamoDB. DynamoDB is a managed key-value and document database. It scales automatically. It is fully serverless.

NoSQL databases are useful for unstructured or semi-structured data. They scale horizontally more easily than SQL databases. This makes them a good fit for large-scale or high-throughput systems.

I have also used other AWS data services. I have used S3 for object storage. S3 stores files and unstructured data. I have used ElastiCache for caching. ElastiCache supports Redis and Memcached.

In my work, I chose SQL when data has clear relationships. I chose SQL when I need strong consistency. I chose NoSQL when the schema is flexible. I chose NoSQL when I need high write throughput or simple key-based lookups. I chose managed AWS services when I want to reduce operational overhead.

SQL and NoSQL differ in several ways.

SQL databases store data in tables. Each table has a fixed schema. Each row must follow the schema. SQL databases support relationships between tables. They use foreign keys and joins. They support ACID transactions. This guarantees strong consistency. Common SQL databases include MySQL, PostgreSQL, and Oracle.

NoSQL databases store data differently. They do not require a fixed schema. Different records can have different fields. NoSQL databases include several types. Document stores like MongoDB store JSON-like documents. Key-value stores like Redis store simple key-value pairs. Column-family stores like Cassandra store wide columns. Graph databases like Neo4j store nodes and relationships. NoSQL databases usually favor availability and scalability over strict consistency.

I use SQL when the data has clear relationships. I use SQL when I need strong consistency. I use SQL for financial transactions. I use SQL for order systems. I use SQL when the schema is stable.

I use NoSQL when the schema changes often. I use NoSQL when I need high write throughput. I use NoSQL when I need simple key-based lookups. I use NoSQL for large-scale systems. I use NoSQL for caching. I use NoSQL for session storage.

In practice, many systems use both. I use SQL for core transactional data. I use NoSQL for caching or high-volume data.

What cloud platform have you used?

Comment: I forgot most of the AWS service. I also did not answer according to their types. The answer is totally not structured.

I have used AWS as my main cloud platform.

Compute. I have used EC2. EC2 provides virtual servers in the cloud. I have used Elastic Beanstalk. Elastic Beanstalk deploys applications without managing the underlying infrastructure. I have used Lambda. Lambda runs code without provisioning servers.

Storage. I have used S3. S3 stores files and unstructured data as objects. I have used EBS. EBS provides block storage attached to EC2 instances.

Database. I have used RDS. RDS is a managed relational database service. I have used DynamoDB. DynamoDB is a managed NoSQL database. I have used ElastiCache. ElastiCache provides in-memory caching with Redis or Memcached.

Networking. I have used VPC. VPC creates an isolated network for AWS resources. I have used Route 53. Route 53 is a DNS service. I have used a load balancer. It distributes traffic across multiple instances.

Security and access. I have used IAM. IAM manages users, roles, and permissions.

CI/CD and deployment. I have used CodePipeline. CodePipeline automates the build and deployment process. I have used ECR. ECR stores Docker container images.

Monitoring. I have used CloudWatch. CloudWatch monitors logs, metrics, and alarms.

What is a serverless service? When would you use servers? When would you use serverless?

Comments: The 3 questions should be together. When I answer what is serverless service, I should also talk about the remaining two questions.

A serverless service does not require me to maintain servers. I do not perform server configuration. I do not manage the infrastructure. I do not manage scaling. AWS handles these tasks. I simply use the service and pay for what I use.

Lambda is a common example. Lambda runs code without provisioning servers. It scales automatically based on demand.

I choose traditional servers when I need more control. EC2 gives me full control over the operating system. EC2 gives me full control over configuration. I use EC2 when my application has custom runtime requirements. I use EC2 when I need long-running processes. I use EC2 when I need predictable performance for steady workloads.

I choose serverless when I want to reduce operational overhead. I use Lambda for short, event-driven tasks. I use Lambda when traffic is unpredictable. I use Lambda when I do not want to manage scaling manually. Serverless also reduces cost for infrequent workloads, since I only pay when the code runs.

In practice, my choice depends on control, workload pattern, and operational effort. I use servers when I need control and consistency. I use serverless when I want simplicity and automatic scaling.

What Angular lifecycle hooks do you know?

Comments: I did not answer this.

Angular components have a lifecycle. Angular provides hooks for each stage. I use these hooks to run code at the right time.

`ngOnChanges` runs first. It runs when an input property changes. It runs before `ngOnInit`. It can run multiple times.

`ngOnInit` runs once. It runs after Angular sets the input properties. I use it to initialize component data. I use it to fetch data from a service.

`ngDoCheck` runs during every change detection cycle. I use it for custom change detection logic. Angular does not detect all changes automatically. This hook lets me catch those changes.

`ngAfterContentInit` runs once. It runs after Angular projects external content into the component. This applies to content passed through `ng-content`.

`ngAfterContentChecked` runs after every check of the projected content.

`ngAfterViewInit` runs once. It runs after Angular initializes the component's view. It also runs after Angular initializes child views. I use it when I need to access the DOM. I use it when I need to access a child component.

`ngAfterViewChecked` runs after every check of the component's view and child views.

`ngOnDestroy` runs once. It runs right before Angular destroys the component. I use it for cleanup. I unsubscribe from observables. I clear timers. I remove event listeners.

In practice, I use `ngOnInit` the most. I use it for initialization logic. I use `ngOnDestroy` next. I use it to prevent memory leaks.

What version of Angular do you use?

Comments: It is using standalone, and it no longer needs NgModule. My wrong answer is to compare Signal with NgModule.

I have mainly worked with Angular 17.

Angular 17 uses standalone components by default, so many applications no longer need `NgModule`.

It also introduces the new `@if` and `@for` control flow syntax, which makes templates cleaner and easier to read.

Angular 17 also supports `@defer`, which loads parts of a page only when they are needed. This helps improve application performance.

I also know that Angular is moving toward signals and zoneless change detection, which reduce unnecessary rendering and improve performance.

